An Industry Oriented Mini Project Report (CS755PC)

MUSIC GENRE CLASSIFICATION

Submitted

in the partial fulfilment of the

requirements for the award of the

degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

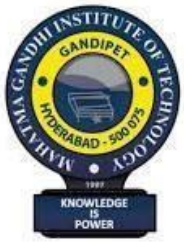
by

Ms. MATTAPARTHY LASYA
(Reg. No. 21261A05H0)

Under the guidance of

Mrs. S. Vijaya Lakshmi

(Assistant Professor, Department of CSE)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

MAHATMA GANDHI INSTITUTE OF TECHNOLOGY

(Affiliated to Jawaharlal Nehru Technological University Hyderabad)

Gandipet, Hyderabad-500075, Telangana(India)

2024-2025

MAHATMA GANDHI INSTITUTE OF TECHNOLOGY

AUTONOMOUS | Affiliated to JNTUH

CERTIFICATE

This is to certify that the report entitled **"Music Genre Classification"** being submitted by **Ms. M. MONISHA SATYA SREE LASYA** bearing Reg. No. **21261A05H0** in partial fulfilment for the award of **Bachelor of Technology (B.Tech.)** in **Computer Science and Engineering to the Department of CSE, Mahatma Gandhi Institute of Technology (A) Hyderabad-500075, T.S**.is a record of Bonafide work carried out by her under our guidance and supervision, at our organization/institution.

The results embodied in this report have not been submitted to any other University or Institute for the award of any degree or diploma.

Signature of Supervisor	Signature of HOD
Mrs. S. Vijaya Lakshmi	**Dr. C. R. K. Reddy**
Assistant Professor	**Professor and HOD**

EXTERNAL EXAMINER

DECLARATION

I hereby declare that the work described in this report, entitled **"**Music Genre Classification**"** which is being submitted by me in partial fulfilment for the award of **Bachelor of Technology** (**B.Tech.)** in Computer Science and Engineering to the **Department of CSE, Mahatma Gandhi Institute of Technology (A)**, Hyderabad-500075, T.S. is the result of investigations carried out by me under the guidance of **Mrs. S. Vijaya Lakshmi** .

The work is original and has not been submitted for any Degree/Diploma of this or any other university.

Place: Hyderabad

Date:

Signature

Name of the Student: **MATTAPARTHY LASYA**

Reg. No: **21261A05H0**

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any task would be incomplete without introducing the people who made it possible and whose constant guidance and encouragement crowns all efforts with success. They have been a guiding light and source of inspiration towards the completion of the project.

I would like to express our sincere thanks to **Dr. G. Chandra Mohan Reddy, Principal MGIT**, for providing the working facilities in college

I would like to express our gratitude to **Dr. C.R.K. Reddy, Professor and HOD**, Department of Computer Science and Engineering, MGIT, for all the timely support and valuable suggestions during the period of our seminar.

I am extremely thankful to **Dr. Shyam Sunder Pabboju, Assistant Professor, Dr. Meera Alphy, Assistant Professor** and **Ms. G. Naga Sujini, Assistant Professor** of Computer Science and Engineering, MGIT, Seminar coordinators for their encouragement and support throughout the Seminar.

I am extremely grateful to our internal guides, **Mrs. S. Vijaya Lakshmi, Assistant Professor** Department of Computer Science and Engineering, MGIT, for their constant encouragement, guidance and moral support throughout the Seminar.

Finally, I would also like to thank all the faculty and staff of CSE Department who helped me directly or indirectly for completing the Minor Project.

Ms. MATTAPARTHY LASYA
(21261A05H0)

ABSTRACT

Music genre classification has become increasingly significant in today's world due to the rapid growth of music tracks both online and offline. To enhance accessibility and organization, it is essential to index these tracks efficiently. Automatic music genre classification plays a crucial role in retrieving music from extensive collections. Most modern classification systems leverage machine learning techniques, which have proven effective for such tasks. This study presents a system comprising ten distinct music genres and employs a deep learning approach for training and classification.

Convolutional Neural Networks (CNNs) are utilized in the proposed system for their superior ability to learn hierarchical patterns in data. Feature extraction, a critical step in audio analysis, is accomplished using Mel Frequency Cepstral Coefficients (MFCCs), which serve as feature vectors for sound samples. The system classifies music genres by analysing features, demonstrating the potential of CNNs to provide accurate and efficient classification.

Keywords: Music Genre Classification, Convolutional Neural Networks, Deep Learning, Feature Extraction, Mel Frequency Cepstral Coefficients.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

CHAPTER 1: INTRODUCTION

Downloading and purchasing music from online music collections has become a part of the daily life of probably a large number of people in the world. The users often formulate their preferences in terms of genre, such as hip hop or pop or disco. However, most of the tracks now available are not automatically classified to a genre. Given the huge size of existing collections, automatic genre classification is important for organization, search, retrieval, and recommendation of music [1]

Music classification is considered a very challenging task due to the selection and extraction of appropriate audio features. While unlabelled data is readily available, music tracks with appropriate genre tags are very few [2] Music genre classification is composed of two basic steps: feature extraction and classification. In the first stage, various features are extracted from the waveform [9] In the second stage, a classifier is built using the features extracted from the training data [8]

There have been many approaches used for the classification of music into different genres. With the huge amount of music available on the internet, there is a strong need for automatic music genre classification [4].Each implementation uses various types of feature extraction. Some can take the timbre, rhythm, etc., as the classifying parameter, while others consider pitch, timbre, and beat [2][9]. The types of features extracted vary depending on the approach taken [7].

The Deep Neural Network (DNN) is widely used in classification problems and is particularly helpful in training on large datasets [3]. This approach has been extended to automatic music genre classification using Convolutional Neural Networks (CNNs) [5]. The features from the music are extracted as Mel Frequency Cepstral Coefficients (MFCCs) for each song. These are obtained by taking the Fourier transforms of the signal, then taking the logarithmic power values, and finally applying the cosine transforms [6][8]. These extracted features then act as the inputs to the neurons for training [1].

For our work, we analyse music from ten various genres. The whole implementation is done in Python programming language. The development of a Music Genre Classifier using deep learning aims to create a robust system capable of accurately predicting music genres [10].

1.1 Problem Statement:

Music genre classification involves identifying the genre of an audio track by analysing its musical elements such as rhythm, pitch, timbre, and tempo. This process begins with audio preprocessing techniques like noise reduction and normalization, followed by feature extraction to highlight key characteristics. These features are used to train machine learning or deep learning models to recognize patterns and classify the genre. The task is challenging because genres often share overlapping features, and variations in instrumentals, vocals, or production quality add complexity. For instance, a jazz song may exhibit rhythmic similarities to blues, making genre boundaries difficult to define. Furthermore, individual interpretations of music genres introduce a subjective element to the classification process.

Modern approaches often transform audio signals into visual representations like spectrograms, enabling the application of computer vision techniques such as Convolutional Neural Networks (CNNs). These models analyse patterns in frequency and time domains to predict genres accurately. This field leverages artificial intelligence to mimic human auditory perception, which naturally discerns genres with ease. Music genre classification not only improves music recommendation systems but also enriches audio analysis applications, making it a crucial and evolving area in AI and machine learning.

1.2 Existing Systems:

Current systems for music genre classification predominantly rely on traditional machine learning techniques or handcrafted feature-based approaches. While these methods have shown some success, they often fall short in capturing the intricate and multi-dimensional aspects of music, limiting their accuracy and scalability in real-world applications.

Traditional Feature-Based Classification: These systems rely on manually extracted features such as tempo, pitch, and rhythm, followed by applying machine learning algorithms like Support Vector Machines (SVMs) or K-Nearest Neighbors (KNN). Although effective for basic classification tasks, their performance is constrained by the quality and relevance of the manually chosen features, making them less suitable for handling diverse and complex datasets.

Rule-Based Systems: Rule-based approaches use predefined heuristics and music theory principles to categorize genres. While these systems can work well for highly structured music, they struggle with genres that overlap or evolve dynamically, resulting in limited adaptability.

Spectrogram Analysis Tools: These methods leverage visual representations of sound, such as spectrograms, for classification. While promising, their effectiveness often depends on manual tuning and expertise in feature extraction, which can be time-intensive and challenging to generalize across datasets.

Despite these advancements, traditional systems face challenges in generalizing across diverse music datasets and capturing the subtle nuances of genres. A CNN-based music genre classification system offers a more robust solution by automating feature extraction and leveraging hierarchical learning to classify music accurately. This approach addresses the limitations of existing methods by providing scalability, adaptability, and enhanced performance for modern music classification needs.

1.3 Proposed System:

This project presents an advanced music genre classification system that harnesses the power of Convolutional Neural Networks (CNNs) to revolutionize the categorization of music tracks. Unlike traditional approaches that rely on manually extracted features and fixed heuristics, the proposed system emphasizes automation, precision, and scalability, making it suitable for diverse music datasets. With an intuitive design, the system aims to simplify the classification process, enabling efficient organization and retrieval of music tracks.

The core of the system is a CNN-based model specifically tailored for analysing audio data. The model goes beyond conventional feature extraction by learning intricate patterns in audio, such as rhythm, timbre, and harmony. Leveraging advanced preprocessing techniques, including the generation of spectrograms and feature extraction using Mel Frequency Cepstral Coefficients (MFCCs), the system ensures robustness and high accuracy across multiple genres.

Designed for adaptability, the proposed system is equipped to handle large-scale datasets and varying audio formats. User-friendly integration with tools like Gradio provides seamless interaction, while ethical considerations, such as intellectual property rights, are embedded into its framework. Rigorous evaluation and testing ensure that the system aligns with industry standards for performance and usability.

By bridging the gap between manual genre classification and the transformative potential of deep learning, this system stands as a versatile and innovative solution. It empowers users, including music enthusiasts and industry professionals, with a reliable tool for efficient music categorization, enhancing the accessibility and organization of music libraries.

1.4 Requirement Specification:

1.4.1 Software Requirements

The proposed system for music genre classification using convolutional neural networks (CNNs) is designed to leverage modern software tools and technologies to ensure efficient preprocessing, feature extraction, and classification of audio data. The specifications include:

- Operating System: Windows 10 (or later versions), macOS, or Linux
- Technology: Python 3.x • Libraries and Frameworks:
  - o TensorFlow and Keras: For building and training CNN models
  - o Librosa: For audio processing and feature extraction
  - o Pandas and NumPy: For data manipulation and analysis
  - o Matplotlib: For visualizing results
  - o Gradio: For creating an interactive user interface
  - o Scikit-learn: For performance evaluation and data splitting
- IDE: Jupyter Notebook or Google Collab

1.4.2 Hardware Requirements

The hardware requirements for the music genre classification system ensure efficient processing of audio datasets and the computational demands of deep learning. The minimum recommended specifications are:

- Processor: Intel Core i5 or equivalent
- RAM: Minimum 8 GB
- Storage: Minimum 250 GB (SSD recommended)
- Graphics Processing Unit (GPU): NVIDIA GPU with CUDA support (optional but recommended for faster training)

1.4.3 Functional Requirements

- Data Preparation: Download and extract the GTZAN dataset for training and testing.
- Feature Extraction: Extract Mel Frequency Cepstral Coefficients (MFCCs) for audio representation.
- Model Training: Train the CNN classifier using the processed dataset.
- Prediction: Classify audio files into predefined genres using the trained model.
- Interactive Testing: Provide predictions through an interactive interface using Gradio.

1.4.4 Non-functional Requirements

- Performance: Ensure high classification accuracy across diverse audio genres.
- Scalability: Support large-scale datasets for training and testing.
- Usability: Provide an intuitive interface for both developers and end users.
- Maintainability: Ensure modular design for ease of updates and improvements.

CHAPTER 2: LITERATURE REVIEW

[1] Shajin Prince, Justin Jojy Thomas, Sharon Jostana J, Kakarla Preethi Priya, Joshua Daniel (2023), "Music Genre Classification using Deep Learning - A Review"

This paper reviews deep learning techniques for music genre classification, focusing on Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and hybrid models like CNN-RNNs. It provides a comprehensive overview of modern architectures and their performance on various datasets. The study highlights the potential of deep learning in music classification but remains largely theoretical, offering limited experimental analysis and real-world implementation.

[2] Bishwomkar Panigrahi, Varun Gandhi, Rahul Bhandari, Shivraj, Kumari Priya (2023), "Harmony in Algorithms: Exploring Music Genre Classification through Machine Learning"

This study employs the GTZAN dataset to extract spectral features, applies CNN for classification, and compares it with traditional algorithms such as SVM, KNN, Naïve Bayes, and Random Forest. CNN achieved high accuracy of 79% and reduced validation losses significantly, outperforming traditional methods. However, the study's scope is limited to the GTZAN dataset, which restricts generalizability to other datasets or music genres.

[3] Nikki Pelchat, Craig M. Gelowitz (2020), "Neural Network Music Genre Classification"

This research utilizes neural networks with spectrogram images as inputs to classify songs into various genres. The study effectively employs spectrograms to capture the timefrequency representation of audio and explores different neural network architectures for classification. However, it shows limited success across different libraries and is highly dependent on the quality and preprocessing of input data.

[4] Mohsin Ashraf, Ikram Ud Din, Jawad Rasheed, Mirsat Yesiltepe (2023), "A Hybrid CNN and RNN Variant Model for Music Classification"

This study combines CNN with RNN to handle both spatial and temporal patterns in audio data, achieving high accuracy on complex datasets. The hybrid model effectively captures

sequential and non-sequential features of audio. However, the approach demands significant computational resources and is prone to overfitting when applied to smaller datasets.

[5] Aru Upadhyay, Anusha Barman, Disha Gupta, Dhirendra Kumar (2023), "Exploring Empirical Mode Decomposition for Music Genre Classification Using Deep Learning"

This research incorporates Empirical Mode Decomposition (EMD) with deep learning models to enhance feature extraction for music genre classification. The method improves classification accuracy for complex audio signals. However, its performance is highly dependent on the quality of input features, which limits its robustness.

Table 2.1 Literature Survey

S. No	Author(s)	Title	Year	Methodology	Merits	Demerits
1	Shajin Prince, Justin Jojy Thomas, Sharon Jostana J, Kakarla Preethi Priya, Joshua Daniel	Music Genre Classification using Deep Learning - A Review	2023	The paper reviews deep learning techniques for music genre classification, focusing on CNNs, RNNs, and hybrid models like CNN-RNNs.	A comprehensive review of modern deep learning architectures discusses their performance across diverse datasets.	The study is primarily theoretical, with limited experimental analysis and real world implementation results.

2	Bishwomkar Panigrahi, Varun Gandhi, Rahul Bhandari, Shivraj, Kumari Priya	Harmony in Algorithms: Exploring Music Genre Classification through Machine Learning	2023	The methodology involves using the GTZAN dataset, extracting spectral features, applying a CNN for classification, and comparing it with SVM, KNN, Naive Bayes, and Random Forest for accuracy and validation loss minimization.	CNN achieved high accuracy of 79%, outperformed traditional methods (SVM, KNN, Naive Bayes, Random Forest). Reduced validation losses significantly	Limited to a single dataset (GTZAN), may not generalize to other music genres or datasets. Other models achieved lower accuracy than CNN.
3	Aru Upadhyay, Anusha Barman, Disha Gupta & Dhirendra Kumar	Exploring Empirical Mode Decomposition for Music Genre Classification using deep learning	2023	Empirical Mode Decomposition (EMD) with deep learning models to improve feature extraction for genre classification	Improved classification accuracy for complex audio signals.	Performance highly dependent on the quality of input features.
4	Mohsin Ashraf, Ikram Ud Din, Jawad Rasheed, Mirsat Yesiltepe	A Hybrid CNN and RNN Variant Model for Music Classification	2023	Combines Convolutional Neural Networks (CNN) with Recurrent Neural Networks (RNN) to handle spatial and temporal patterns in audio data.	The method effectively handles both sequential and non-sequential audio features, achieving high accuracy on complex datasets.	The approach requires significant computational power and is prone to overfitting on small datasets.

5	Nikki Pelchat, Craig M. Gelowitz	Neural Network Music Genre Classification	2020	Utilized neural networks (NNs) with spectrogram images as inputs to classify songs into musical genres.	The study effectively utilizes spectrograms to capture time-frequency representations of audio and explores various neural network techniques for music classification.	The approach shows limited success across different libraries and is highly dependent on the quality and preprocessing of input data.

CHAPTER 3: DESIGN METHODOLOGY

3.1 System Architecture

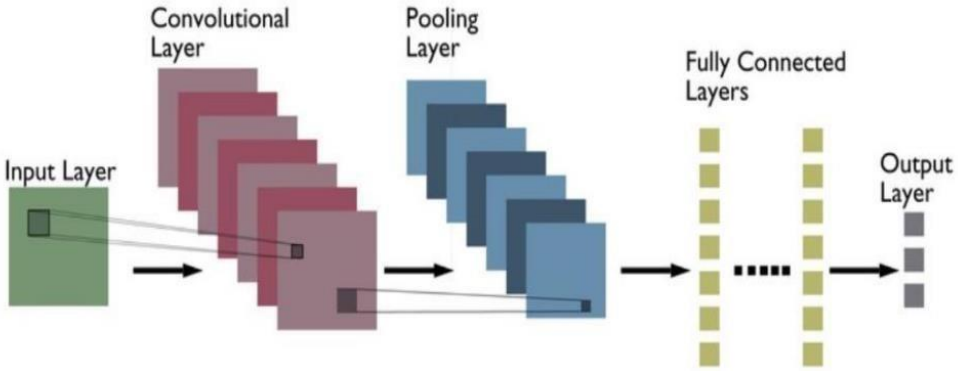


Figure 3.1: System Architecture of Music Genre Classification

Convolutional layers play a critical role in audio data processing by extracting essential features like rhythms, pitches, and timbres. These layers identify local patterns in the spectrograms, enabling the model to understand the structure and texture of the audio. Following this, pooling layers simplify the extracted features through down-sampling techniques such as max-pooling or average pooling. This step retains the most significant information while reducing computational complexity and mitigating overfitting.

Activation layers introduce non-linearity into the model, allowing it to capture complex patterns and relationships within the audio data. This non-linearity is crucial for modelling intricate musical characteristics that are not linearly separable.

The flatten layer takes the multi-dimensional feature maps from the convolutional and pooling layers and reshapes them into a single-dimensional array. This transformation is essential for preparing the data for input into fully connected layers, which require a fixed-size vector.

Finally, dense layers, also known as fully connected layers, process the flattened features to make predictions about the audio. These layers combine the extracted features to output the final classification, such as the genre of music, by leveraging learned weights and biases. Together, these layers create a robust pipeline for audio data analysis and classification.

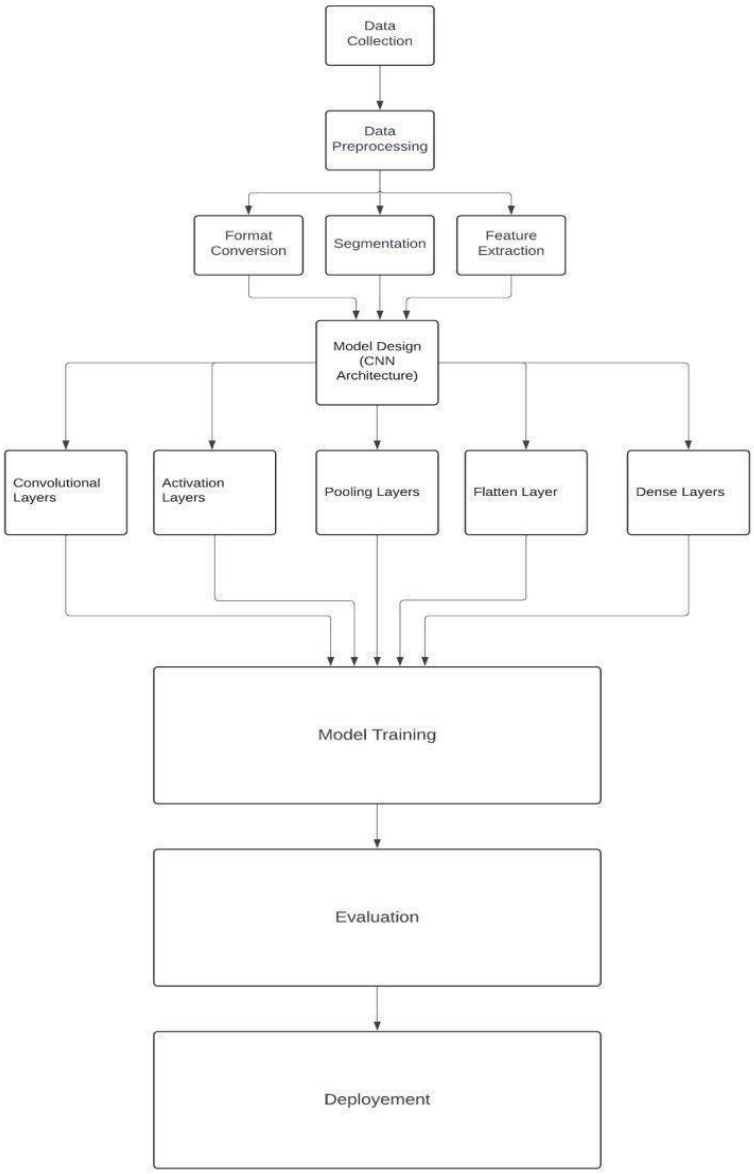


Figure 3.1 Flow Diagram of Music Genre Classification

The above picture depicts a flow diagram outlining the system architecture which shows the various phases of the system: input, data pre-processing, model architecture and output.

3.2 Methodology

3.2.1 Data Collection

Data collection is the foundational step in building a music genre classifier. It involves gathering a comprehensive dataset of audio samples, each labelled with its corresponding genre. The dataset should encompass a wide variety of music styles to ensure diversity and representativeness, enabling the model to generalize well across different genres. High quality datasets, such as the GTZAN dataset, are commonly used for this purpose. Each audio file should meet specific technical standards, such as consistent sampling rates and bit depths, to maintain uniformity. Proper labelling of genres ensures accurate training and evaluation of the classifier. The diversity of the dataset is crucial for enhancing the model's ability to distinguish subtle differences across various music genres effectively.

3.2.2 Feature Engineering

Preprocessing is a crucial step in preparing audio data for the CNN model in music genre classification. This stage ensures that all audio files are in a consistent format, such as a uniform sampling rate and bit depth, facilitating smooth processing and analysis. The process begins with format conversion, where audio files are standardized to ensure uniformity across the dataset. Next, segmentation divides the audio files into shorter segments or frames, making it easier for the model to analyse distinct parts of the audio.

The most critical step is featuring extraction, where relevant features are derived from these segments. Features like Mel-Frequency Cepstral Coefficients (MFCCs) capture the timbral characteristics, Spectral Flux measures the rate of change in frequency, and Chroma features represent the harmonic content. These features collectively encapsulate rhythm, timbre, and harmony, enabling the CNN model to learn patterns and classify audio into specific music genres effectively.

3.2.3 Model Design (CNN Architecture)

The Model Design for the music genre classifier revolves around building a robust CNN architecture tailored for genre classification. This design leverages multiple layers, each serving a specific role in processing audio features and enabling accurate classification. The architecture begins with Convolutional Layers, which apply learnable filters to the input

data, extracting essential features such as patterns in frequency and time domains. These features represent critical aspects like rhythm, timbre, and harmony. Following these,
Pooling Layers down-sample the extracted feature maps, reducing the data's complexity and computational load while enhancing the model's ability to generalize across diverse inputs.
To introduce non-linearity and enable the model to learn complex relationships between features, Activation Layers are incorporated. A Flatten Layer follows, converting the multidimensional feature maps into a one-dimensional vector suitable for classification. This transformation prepares the data for the subsequent Dense Layers, which are fully connected layers that analyse the extracted features and perform the final classification task. The architecture concludes with an output layer, sized according to the number of target genres. This systematic layering ensures the CNN model effectively processes audio data, captures intricate musical patterns, and predicts genres with high accuracy.

3.2.4 Model Training

Training the CNN model involves using the prepared dataset of labelled audio samples to teach the model to classify music genres accurately. During this process, the model iteratively adjusts its weights and biases through backpropagation and optimization algorithms, such as stochastic gradient descent (SGD). The objective is to minimize the classification error by reducing the difference between the predicted and actual genre labels. Training typically involves multiple epochs, where the dataset is repeatedly passed through the model, enabling it to learn intricate patterns and features in the data. This process ensures improved accuracy and reliability in genre classification.

3.2.5 Evaluation

Evaluation assesses the model's performance on unseen data by testing its ability to generalize and accurately classify new audio samples. Metrics like accuracy, precision, recall, and F1-score measure the effectiveness of the trained CNN model.

3.2.5 Deployment (Optional)

The trained model can be integrated into real-world applications like music streaming platforms, enabling automatic genre classification of new audio data. This facilitates personalized recommendations, enhanced user experience, and efficient organization of vast music libraries.

3.3 UML MODELS

UML is a standard language used to create software blueprints by specifying, visualizing, constructing, and documenting system designs. It helps developers communicate and model software structures effectively.

3.3.1 Use Case Diagram

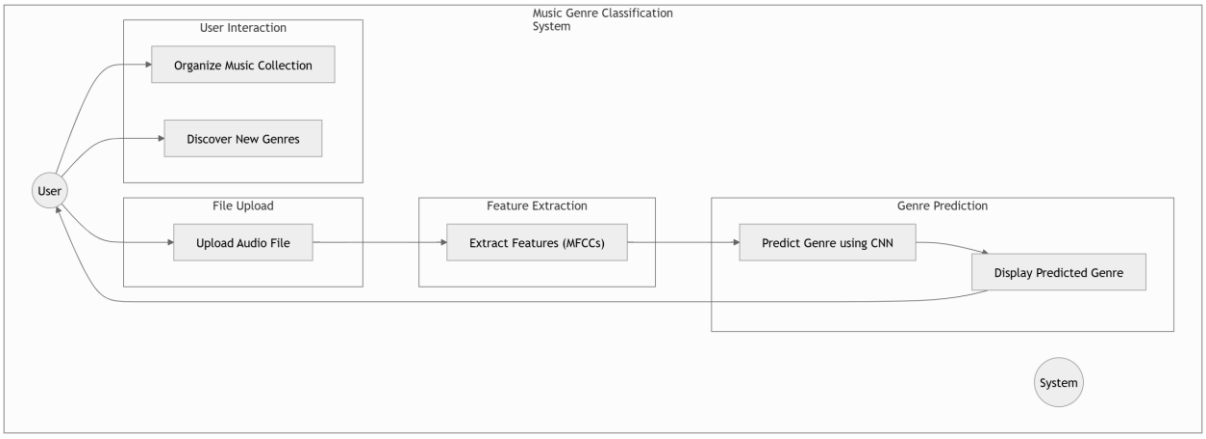


Figure 3.3 Use case diagram of Music Genre Classification

Figure 3.3 represents how the entities and relationships work. Here the actor is the user who uploads an audio file to the application which then returns prediction of the genre of uploaded audio to user.

3.3.2 Class Diagram

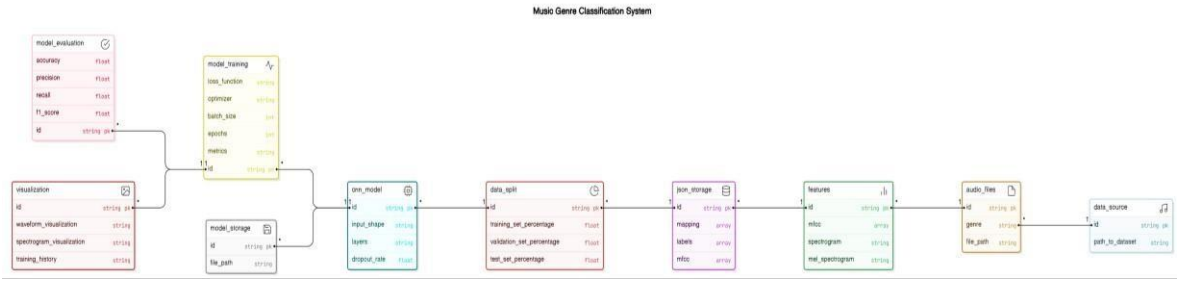


Figure 3.3 Class diagram of Music Genre Classification

The above figure 3.3 represents the structure of the Music Genre Classification System. It outlines a comprehensive framework for a music genre classification system. It begins with data source, storing the dataset path, linked to audio files containing genres and file paths. Extracted features like MFCCs, spectrograms, and Mel spectrograms are stored in features, with mappings and labels managed in Json storage. The data split entity defines training, validation, and test percentages. The CNN model stores architecture details, while model training tracks hyperparameters like optimizer and epochs. Performance metrics like accuracy and F1-score are stored in model evaluation, and visualizations are handled in visualization.

3.3.3 Sequence Diagram

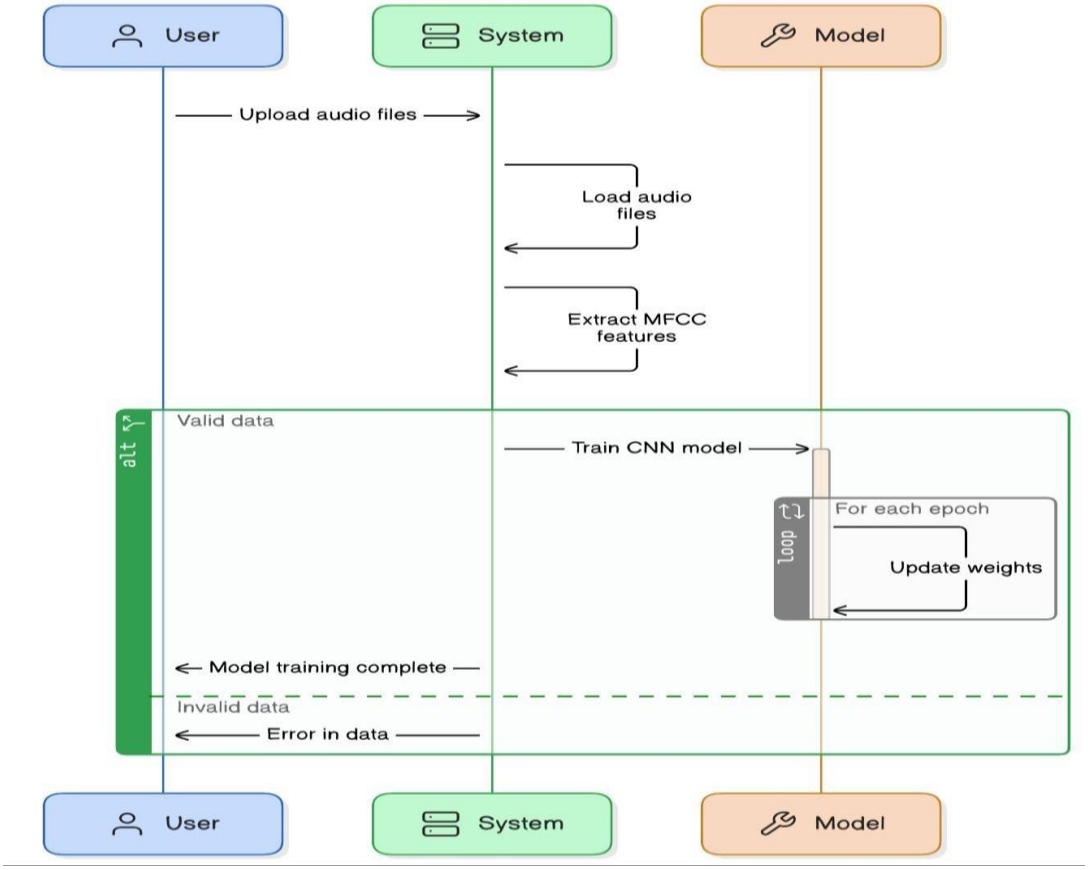


Figure 3.3 Sequence Diagram of Music Genre Classification

The above figure 3.3 The sequence diagram represents a music genre classification workflow. The user uploads audio files, which the system processes by loading the files and extracting MFCC (Mel Frequency Cepstral Coefficients) features. If the data is valid, the system trains a CNN (Convolutional Neural Network) model, iteratively updating weights for each epoch. Once training is complete, the model is ready. In cases of invalid data, errors are flagged and reported to the user. The process ensures an efficient pipeline for classification tasks.

CHAPTER 4: IMPLEMENTATION AND RESULTS

The implementation involves using a Convolutional Neural Network (CNN) model trained on Music Dataset to predict music genres. The GTZAN dataset consists of a collection of 10 genres with 100 audio files each, all having a length of 30 seconds and a visual representation for each audio file.

4.1 Data Collection

This project uses GTZAN data set. The data set consists:

- Genre's original - A collection of 10 genres with 100 audio files each, all having a length of 30 seconds (the famous GTZAN dataset, the MNIST of sounds)

- Images original - A visual representation for each audio file. One way to classify data is through neural networks. Because NNs (like CNN, what we will be using today) usually take in some sort of image representation, the audio files were converted to Mel Spectrograms to make this possible.

- 2 CSV files - Containing features of the audio files. One file has for each song (30 seconds long) a mean and variance computed over multiple features that can be extracted from an audio file. The other file has the same structure, but the songs were split before into 3 seconds audio files (this way increasing 10 times the amount of data we fuel into our classification models). With data, more is always better.

The GTZAN dataset is used for classification task. It consists of 1000 audio tracks, each 30 seconds long, sampled at 22050Hz.

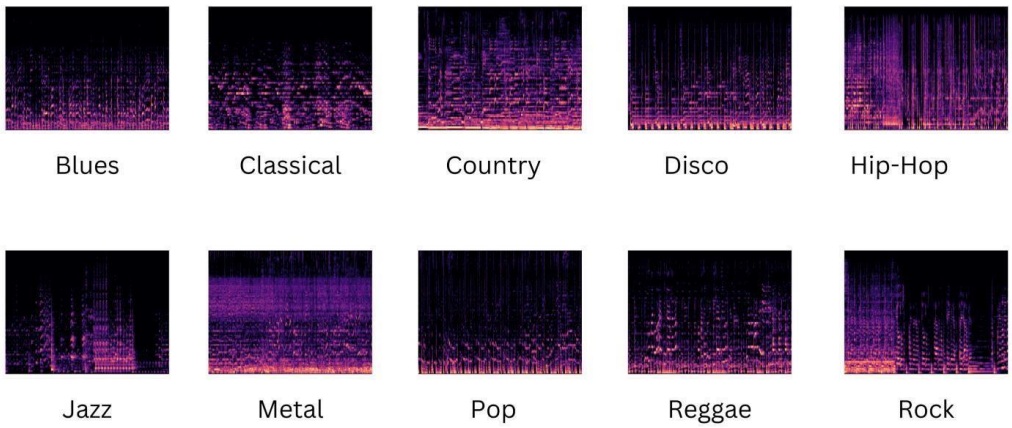


Figure 4.1 Example Dataset

4.2 Data Preprocessing

Preprocessing focuses on preparing audio data for analysis by the CNN model, ensuring consistency and extracting meaningful features.

1. **Format Conversion:** Firstly, we ensure that all audio files in our dataset have a consistent format. This includes standardizing parameters such as sampling rate and bit depth. By ensuring consistency in these aspects, we create a level playing field for our model to learn from the data.
2. **Segmentation:** Next, we divide the audio files into shorter segments, often referred to as frames. This segmentation allows us to Analyse smaller portions of the audio data at a time, making the processing more manageable and efficient. Additionally, it enables us to capture the temporal characteristics of the audio, which are crucial for genre classification.
3. **Feature Extraction:** Once we have segmented the audio files, we extract relevant features from these segments. These features serve as inputs to our CNN model and capture essential aspects of the audio, such as rhythm, timbre, and harmony. Some commonly used features include Mel Frequency Cepstral Coefficients (MFCCs), which represent the spectral envelope of the audio signal, Spectral Flux, which measures the change in spectral content over time, and Chroma features, which represent the pitch content of the audio in a compact form

4.3 Model Architecture

The Convolutional Neural Network (CNN) architecture for music genre classification is meticulously designed to extract and analyse features from audio data, culminating in accurate predictions. The architecture includes the following key components:

Convolutional Layers:
These layers serve as the primary feature extractors, applying learnable filters to the audio data to capture essential patterns. Each filter specializes in detecting specific attributes, such as rhythm or frequency components, from the input.

Pooling Layers:
Pooling layers are introduced to down sample the feature maps generated by the convolutional layers. This operation reduces the spatial dimensions of the data, thereby simplifying computations, improving efficiency, and aiding generalization by retaining the most significant features.

Activation Layers:
Activation functions, such as ReLU (Rectified Linear Unit), are employed to introduce nonlinearity into the model. This enables the network to learn and represent complex relationships within the data, crucial for distinguishing between similar genres.

Flatten Layer:
The multi-dimensional output from the convolutional and pooling layers is transformed into a one-dimensional vector, making it suitable for classification in subsequent dense layers.

Dense Layers:
Fully connected layers analyse the extracted features, making genre predictions. The final dense layer outputs probabilities for each genre class, corresponding to the number of genres in the dataset.

4.4 Model Training

The training process of the Convolutional Neural Network (CNN) model for music genre classification begins by fitting the model to the pre-processed audio data using the fit method. The fit method processes training batches containing feature arrays (e.g., MFCCs) and their corresponding genre labels. The model iterates through several epochs to optimize its weights based on the loss function, allowing it to improve its predictions progressively.

The model is trained for a specific number of epochs (e.g., 20), where each epoch represents a complete pass through the entire training dataset. During each epoch, forward propagation is performed: the extracted features are passed through the CNN layers, and the predicted genre outputs are compared to the actual labels to compute the loss. The loss function, such as categorical cross-entropy, measures the difference between predicted probabilities and true genre labels.

After forward propagation, backpropagation calculates the gradients of the loss with respect to the model's weights. These gradients are utilized by an optimizer (e.g., Adam) to adjust the weights and minimize the loss function. The Adam optimizer is particularly effective as it dynamically adapts the learning rate using the mean and variance of gradients, ensuring efficient and stable convergence.

Throughout the training process, performance metrics like accuracy are monitored at the end of each epoch to assess how well the model classifies genres. To prevent overfitting, techniques like dropout layers, data augmentation, or early stopping might be employed. After completing the training, the model will have optimized weights that enable it to classify audio segments into one of the predefined music genres with high accuracy.

4.5 Results

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 214, 11, 32)	320
max_pooling2d (MaxPooling2D)	(None, 107, 6, 32)	0
batch_normalization (BatchNormalization)	(None, 107, 6, 32)	128
conv2d_1 (Conv2D)	(None, 105, 4, 32)	9,248
max_pooling2d_1 (MaxPooling2D)	(None, 53, 2, 32)	0
batch_normalization_1 (BatchNormalization)	(None, 53, 2, 32)	128
conv2d_2 (Conv2D)	(None, 52, 1, 32)	4,128
max_pooling2d_2 (MaxPooling2D)	(None, 26, 1, 32)	0
batch_normalization_2 (BatchNormalization)	(None, 26, 1, 32)	128
flatten (Flatten)	(None, 832)	0
dense (Dense)	(None, 64)	53,312
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 10)	650

Figure 4.5 Model Summary

In the above figure 4.5.1 Convolutional layers extract essential features from audio data, identifying patterns like frequency and time. Pooling layers simplify the data by reducing complexity while retaining key features. Activation layers introduce non-linear relationships, enabling the model to learn intricate patterns. The flatten layer reshapes the extracted features into a one-dimensional array, making them suitable for classification. Finally, dense layers use this processed data to make accurate genre predictions, acting as the decision-making component of the network.

```
113/113 ━━━━━━━━━━━━━━━━━━━━ 15s 105ms/step - accuracy: 0.1693 - loss: 2.7631 - val_accuracy: 0.2336 - val_loss: 2.3203
Epoch 2/30
113/113 ━━━━━━━━━━━━━━━━━━━━ 22s 122ms/step - accuracy: 0.3301 - loss: 2.0064 - val_accuracy: 0.3615 - val_loss: 1.8529
Epoch 3/30
113/113 ━━━━━━━━━━━━━━━━━━━━ 11s 98ms/step - accuracy: 0.3869 - loss: 1.7818 - val_accuracy: 0.4160 - val_loss: 1.6532
Epoch 4/30
113/113 ━━━━━━━━━━━━━━━━━━━━ 13s 113ms/step - accuracy: 0.4577 - loss: 1.6137 - val_accuracy: 0.4572 - val_loss: 1.5205
Epoch 5/30
113/113 ━━━━━━━━━━━━━━━━━━━━ 14s 120ms/step - accuracy: 0.4913 - loss: 1.4681 - val_accuracy: 0.4983 - val_loss: 1.4528
Epoch 6/30
113/113 ━━━━━━━━━━━━━━━━━━━━ 21s 128ms/step - accuracy: 0.5178 - loss: 1.3860 - val_accuracy: 0.4972 - val_loss: 1.3813
Epoch 7/30
113/113 ━━━━━━━━━━━━━━━━━━━━ 19s 119ms/step - accuracy: 0.5378 - loss: 1.2920 - val_accuracy: 0.5184 - val_loss: 1.3363
Epoch 8/30
113/113 ━━━━━━━━━━━━━━━━━━━━ 18s 101ms/step - accuracy: 0.5843 - loss: 1.1982 - val_accuracy: 0.5395 - val_loss: 1.2759
Epoch 9/30
113/113 ━━━━━━━━━━━━━━━━━━━━ 22s 114ms/step - accuracy: 0.5848 - loss: 1.1726 - val_accuracy: 0.5628 - val_loss: 1.2523
Epoch 10/30
113/113 ━━━━━━━━━━━━━━━━━━━━ 21s 122ms/step - accuracy: 0.6206 - loss: 1.0892 - val_accuracy: 0.5640 - val_loss: 1.2138
Epoch 11/30
113/113 ━━━━━━━━━━━━━━━━━━━━ 21s 122ms/step - accuracy: 0.6261 - loss: 1.0577 - val_accuracy: 0.5806 - val_loss: 1.1985
Epoch 12/30
113/113 ━━━━━━━━━━━━━━━━━━━━ 12s 108ms/step - accuracy: 0.6451 - loss: 1.0467 - val_accuracy: 0.5829 - val_loss: 1.1670
Epoch 13/30
113/113 ━━━━━━━━━━━━━━━━━━━━ 22s 122ms/step - accuracy: 0.6654 - loss: 0.9766 - val_accuracy: 0.5829 - val_loss: 1.1631
Epoch 14/30
113/113 ━━━━━━━━━━━━━━━━━━━━ 20s 122ms/step - accuracy: 0.6869 - loss: 0.9237 - val_accuracy: 0.6073 - val_loss: 1.1334
Epoch 15/30
113/113 ━━━━━━━━━━━━━━━━━━━━ 21s 122ms/step - accuracy: 0.6646 - loss: 0.9420 - val_accuracy: 0.6073 - val_loss: 1.1213
Epoch 16/30
113/113 ━━━━━━━━━━━━━━━━━━━━ 20s 115ms/step - accuracy: 0.6884 - loss: 0.8916 - val_accuracy: 0.5973 - val_loss: 1.1359
Epoch 17/30
113/113 ━━━━━━━━━━━━━━━━━━━━ 18s 96ms/step - accuracy: 0.6963 - loss: 0.8778 - val_accuracy: 0.6129 - val_loss: 1.0870
Epoch 18/30
113/113 ━━━━━━━━━━━━━━━━━━━━ 14s 121ms/step - accuracy: 0.7053 - loss: 0.8291 - val_accuracy: 0.6118 - val_loss: 1.0791
```

Figure 4.5 Model Epochs

In the above figure 4.5.2 it shows the model epochs. In each epoch the training dataset is passed once through the model, having multiple epochs benefits the model as it can learn the complex patterns in the images.

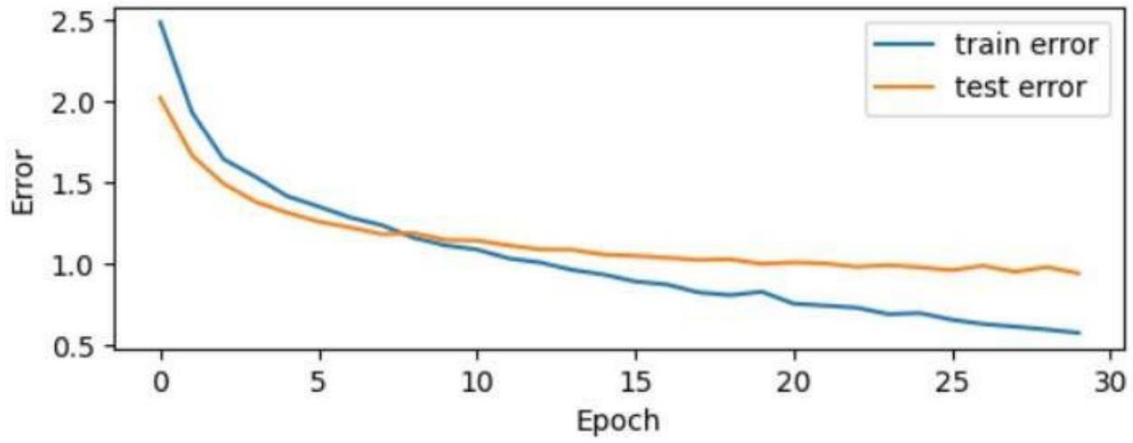


Figure 4.5 Training and Validation loss

The above figure 4.5.3 The graph demonstrates a consistent decrease in both training and testing errors over 30 epochs, showcasing the model's learning progress. While the test error stabilizes, the training error continues to improve, indicating the model's strong capacity to fit the data.

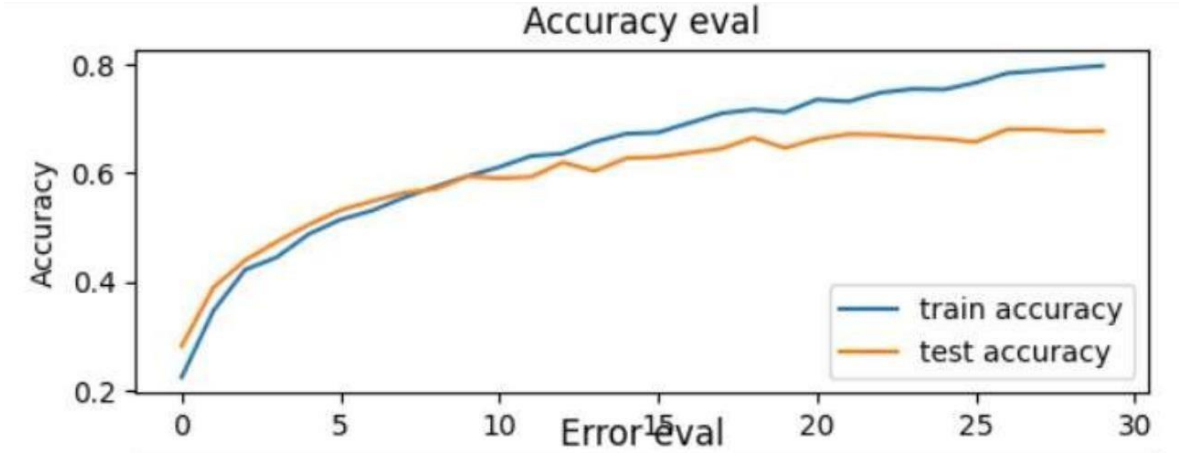


Figure 4.5 Training and Validation accuracy

The above figure 4.5.4 The graph shows a steady improvement in both training and testing accuracy over 30 epochs, highlighting the model's learning progress. While the training accuracy continues to increase, the test accuracy stabilizes, demonstrating the model's ability to generalize effectively.

4.6 User Interface

The code creates an interface for predicting music genre from uploaded audio files. Users upload a file, and the system analyses its content using features like MFCCs. These features are then used by a pre-trained CNN model to predict the genre. The predicted genre, based on the most frequent occurrences across segments, is displayed as text output. This user friendly interface is useful for organizing music collections or discovering new genres.

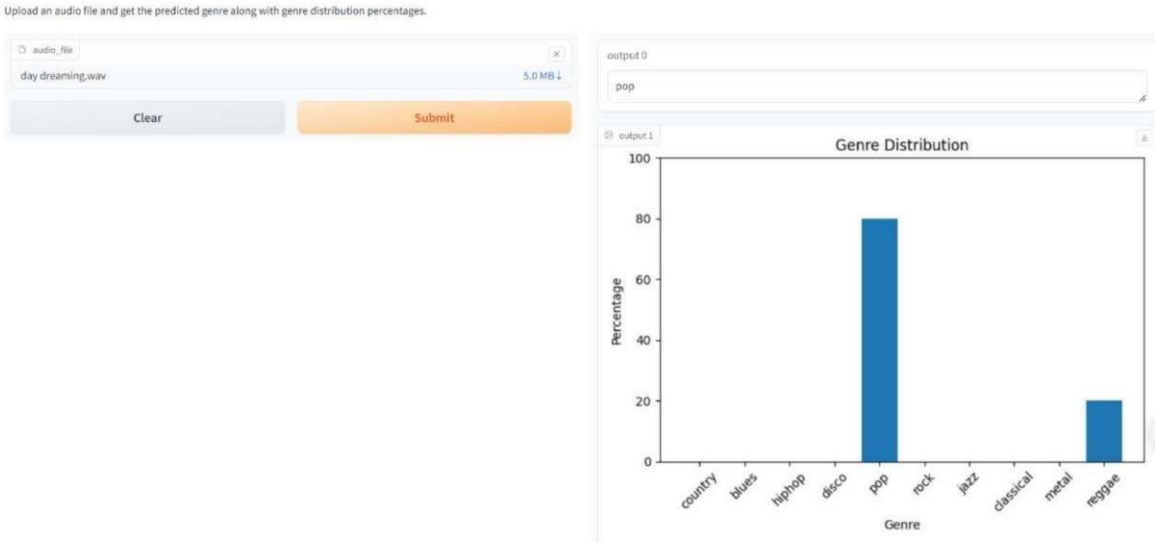


Figure 4.6 Home Page of User Interface

4.6.1 App Structure

The application is defined in the Music Genre Classifier App, which contains the logic for loading the model, preprocessing audio files, and rendering various pages. There are three main sections of the app:

1. Home Page: Provides an introduction to the app and its ability to classify music genres. It explains the importance of genre classification in music personalization and discovery. The page also includes examples of popular genres like Pop, Rock, Jazz, and Classical to give users an idea of the model's capability.
2. Upload Audio Page: Allows users to upload an audio file in formats like .wav. The app preprocesses the audio by extracting features like MFCCs, Chroma, and Spectral Flux. These features are passed to the trained CNN model for classification. The predicted genre (e.g., "Pop") and a probability distribution of other genres are displayed along with a bar chart visualization.

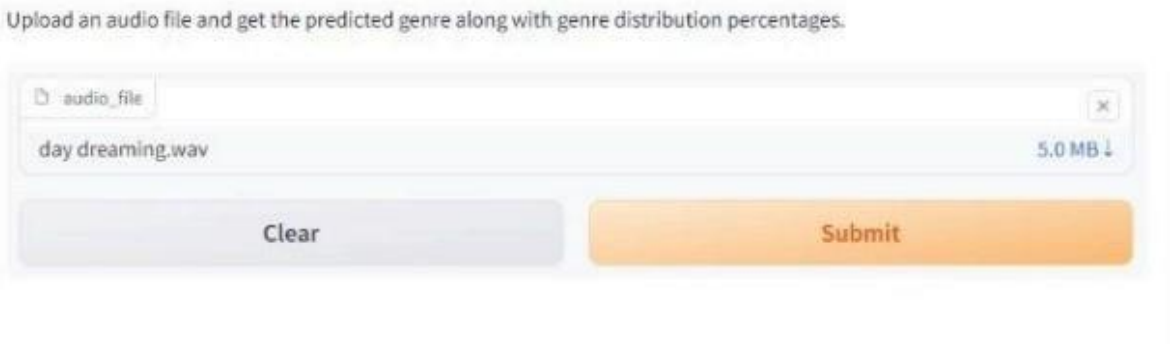


Figure 4.6 Upload Page of User Interface

3. About Page: Provides information about the app and its underlying technology. It explains the frameworks used, such as Streamlit for the frontend, TensorFlow/ Keras for the model, and libraries like Librosa for feature extraction. It also highlights the CNN model's design and its accuracy in classifying music genres.

4.6.2 Model Loading and Prediction

The application uses the Keras API to load the pre-trained model from a saved file (music_cnn.h5). The model is a deep learning-based Convolutional Neural Network (CNN) that has been trained to classify music genres from audio data. When an audio file is uploaded, its features (such as MFCCs) are extracted and passed to the model for classification. The model's output is then used to predict the dominant genre and provide insights into the genre distribution within the audio file.

Result Display:

The code creates an interface for predicting music genre from uploaded audio files. Users upload a file, and the system analyses its content using features like MFCCs. These features are then used by a pre-trained CNN model to predict the genre. The predicted genre, based on the most frequent occurrences across segments, is displayed as text output. This userfriendly interface is useful for organizing music collections or discovering new genres.

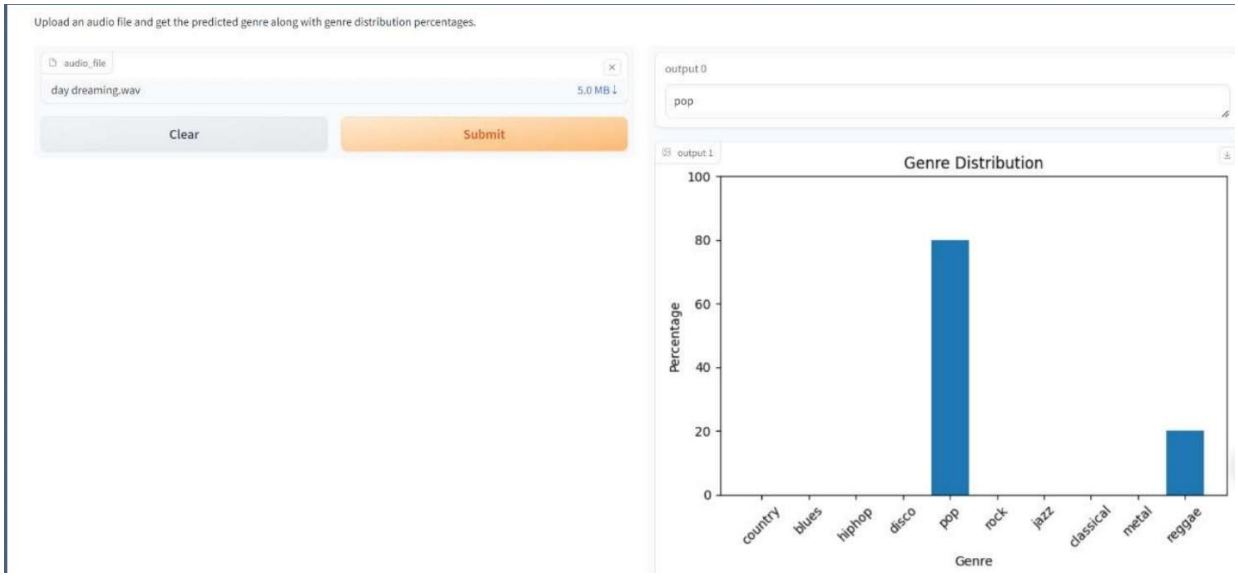


Figure 4.6 Prediction based on uploaded audio

4.6.3 Technology Stack

The app is built using the following technologies:

- **Gradio**: For creating an interactive and user-friendly interface for genre prediction.
- **TensorFlow and Keras**: For training and deploying the deep learning model.
- **Librosa**: For audio processing and feature extraction.

4.6.4 Navigation and Layout

The app's interface allows users to upload an audio file for genre classification and view the results in a structured format. The layout is designed to ensure user-friendliness, with a clear **upload button** for selecting audio files and a **submit button** for processing.

The output section displays two key elements:

• The predicted genre as texshown prominently for easy interpretation.

• A bar chart visualizing the genre distribution percentages, providing detailed insights into the audio file's composition.

The layout is clean and optimized for interaction, effectively utilizing space to present both input and output elements in an accessible manner.

4.7 Testing

4.7.1 Unit Testing

Unit testing involves testing individual components of the system, such as audio preprocessing functions (e.g., feature extraction, normalization) and the CNN model layers. The goal is to verify that each component works correctly before they are integrated into the complete music genre classification system.

For instance, the audio preprocessing functions are unit tested to ensure they handle tasks like extracting Mel-frequency cepstral coefficients (MFCCs), normalizing feature values for consistency, and segmenting the audio correctly for analysis. These tests ensure that the audio data is accurately prepared for input into the CNN.

Similarly, the CNN model layers, including convolutional layers, pooling layers, and fully connected layers, are tested to ensure they process the input data as expected. For example, the convolutional layers should correctly extract relevant audio features (such as rhythm, pitch, or timbre) from the input MFCCs, the pooling layers should correctly reduce the dimensionality while preserving important features, and the fully connected layers should properly classify the features into corresponding music genres.

These unit tests validate that each layer functions correctly and produces the expected outputs, helping to ensure the overall accuracy and reliability of the model for predicting music genres.

4.7.2 Functional Testing

Functional testing is performed to ensure that the system functions as expected. This includes:

- **Valid Input:** Ensuring that valid audio files (properly formatted .wav files of 30 seconds duration) are accepted by the model for processing.
- **Invalid Input:** Ensuring that invalid inputs, such as incorrectly formatted audio files, unsupported file types, or files exceeding the required duration, are rejected with appropriate error messages.
- **Functions:** Verifying that the system's core functionalities, such as feature extraction (MFCC generation), classification, and prediction, work as intended without errors.

- **Output:** Confirming that the predicted genre labels and genre distribution percentages are accurate and consistent with the expected classifications for the provided audio data.

4.7.3 System Testing

System testing is carried out to validate the entire integrated system, from data input (audio file upload) to model prediction (classification of music genres). The system is tested to ensure that the integrated CNN model performs accurately and generates predictions without errors. This also includes testing the user interface to ensure that users can upload audio files, view the predicted genre, and analyse the genre distribution as expected.

4.7.4 Performance Testing

Performance testing ensures that the system can handle the expected load and deliver results within a reasonable time. The model's inference time is tested to confirm that predictions are made in a timely manner, allowing for real-time or near-real-time usage. The system's scalability is also tested to evaluate how it handles larger datasets of audio files or multiple simultaneous audio uploads without compromising performance.

4.7.5 Integration Testing

Integration testing focuses on testing the interaction between various components of the system, such as the audio preprocessing pipeline, genre prediction model, and the user interface. The goal is to ensure that data flows seamlessly from one component to another, from the uploaded audio file to the feature extraction, and finally to the genre classification model. This ensures that the integrated system performs correctly, with no errors occurring during the interaction between components, such as the feature extraction, model prediction, and result display.

4.7.6 Acceptance Testing

User Acceptance Testing (UAT) is conducted to ensure the music genre classification system meets the functional requirements and is ready for deployment. End-users (e.g., music enthusiasts or industry professionals) will test the system to ensure it accurately classifies audio files and provides a user-friendly interface. This test ensures the system meets their expectations and requirements for practical use, including ease of use in uploading audio files, displaying predicted genres, and visualizing genre distribution accurately.

CHAPTER 5: CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

In this project, a deep learning-based approach was developed for classifying music genres from audio data using a Convolutional Neural Network (CNN) model. The model processed spectrogram representations of audio signals to extract meaningful features and classify genres. Experimental results yielded a test accuracy of **67.56%**, demonstrating the model's capability to identify distinct musical patterns.

While the achieved accuracy highlights the potential of CNN architectures for music genre classification, there is room for improvement. Enhancing the dataset's size and diversity, incorporating advanced techniques like transfer learning or hybrid models (e.g., CNN-RNN), and fine-tuning hyperparameters could further boost performance.

This classifier underscores the effectiveness of deep learning for music analysis and its potential for applications in music recommendation systems, audio content classification, and digital media management. The findings provide a strong foundation for future research and innovation in automated music genre classification.

5.2 Future Scope

- **Dataset Expansion and Diversity:** Expanding the dataset with a broader collection of audio tracks from various genres, cultures, and languages will enhance the model's performance and reduce biases. A diverse dataset ensures the classifier's ability to generalize effectively across different musical styles and regions, improving its global applicability. Incorporating real-world noisy data will further strengthen the model’s robustness for practical use.
- **Incorporation of Advanced Features:** Integrating temporal and harmonic features such as Mel-frequency cepstral coefficients (MFCCs) or chroma features, alongside spectral data, can improve the model’s ability to distinguish subtle differences between genres. Leveraging hybrid models combining CNNs with RNNs or attention mechanisms can capture sequential dependencies in music, enhancing classification accuracy.

- **Edge Computing for Real-Time Classification:** Deploying the music genre classifier on local devices like smartphones or embedded systems enables real-time classification without relying on cloud resources. This reduces latency, enhances privacy, and makes the system more accessible for music enthusiasts, DJs, and streaming platforms in offline scenarios.
- **Mobile Application Integration:** Developing a mobile application with the genre classifier allows users to analyze audio tracks on-the-go. Features like playlist organization, genre-based recommendations, and personalized music suggestions can improve user experience and promote wider adoption. This integration can also facilitate its use in applications like music education and curation tools.
- **Cross-Platform Integration:** Extending the system to integrate with popular music streaming platforms such as Spotify or Apple Music can enable genre-based content filtering and recommendations. Such applications can enhance user satisfaction by providing tailored listening experiences based on advanced classification techniques.

REFERENCES

[1] Shajin Prince, Justin Jojy Thomas, Sharon Jostana J., Kakarla Preethi Priya, Joshua Daniel, "Music Genre Classification using Deep Learning - A Review", 2023.

[2] Bishwomkar Panigrahi, Varun Gandhi, Rahul Bhandari, Shivraj, Kumari Priya, "Harmony in Algorithms: Exploring Music Genre Classification through Machine Learning", 2023.

[3] Nikki Pelchat, Craig M. Gelowitz, "Neural Network Music Genre Classification", 2020.

[4] Mohsin Ashraf, Ikram Ud Din, Jawad Rasheed, Mirsat Yesiltepe, "A Hybrid CNN and RNN Variant Model for Music Classification", 2023.

[5] Aru Upadhyay, Anusha Barman, Disha Gupta, Dhirendra Kumar, "Exploring Empirical Mode Decomposition for Music Genre Classification Using Deep Learning", 2023.

[6] J. Kim, H. Hwang, J. Cho, S. Kim, "Deep Learning-Based Music Genre Classification with Audio Feature Extraction", International Conference on Artificial Intelligence and Data Processing, 2021.

[7] L. Smith, R. Patel, "A Comparative Study of Machine Learning Algorithms for Music Genre Classification", International Journal of Music Computing, vol. 15, pp. 245-259, 2022.

[8] M. Sharma, V. Kumar, R. Kumar, "Deep Neural Networks for Music Genre Classification Using Audio Spectrograms", 2020 IEEE International Conference on Audio and Music, pp. 8994, 2020.

[9] A. Desai, V. Rao, "Music Genre Classification: A Review of Classical and Deep Learning Approaches", 2021 International Conference on Machine Learning and Signal Processing (MLSP), pp. 113-120, 2021.

[10] S. Agarwal, R. Yadav, P. Verma, "Hybrid Approaches in Music Genre Classification: An Analysis of CNN and LSTM Models", International Journal of Music Engineering and Technology, vol. 4, pp. 76-82, 2023.

APPENDIX A: SOURCE CODE

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

import os, json, math, librosa

import IPython.display as ipd
import librosa.display

import tensorflow as tf
import tensorflow.keras as keras

from tensorflow.keras import Sequential
from tensorflow.keras.layers import Conv2D

import sklearn.model_selection as sk

from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
from sklearn.metrics import accuracy_score, precision_recall_fscore_support



# Getting Genres from folder name



# MUSIC = '../input/gtzan-dataset-music-genre-classification/Data/genres_original/'
MUSIC = '/content/drive/MyDrive/Data/genres_original'
music_dataset = [] # File locations for each wav file
genre_target = [] #
for root, dirs, files in os.walk(MUSIC):
    for name in files:
        filename = os.path.join(root, name)
        if filename != '/Data/genres_original/jazz/jazz.00054.wav':
        # if filename != 'Data/genres_original/metal/metal.00000.wav':
            music_dataset.append(filename)
            genre_target.append(filename.split("/")[-2])
```

```
# Displaying genres
print(set(music_dataset))
print(set(genre_target))


# Testing Audio Files

audio_path = music_dataset[50]
# img_path = './data/images_original/blues/blues00011.png'


x , sr = librosa.load(audio_path)



librosa.load(audio_path, sr=None)

ipd.Audio(audio_path)



# Visualizing Audio File as a waveform
# plt.figure(figsize=(16, 5))
plt.figure(figsize=(16, 5))
librosa.display.waveshow(x, sr=sr)



# Visualizing audio file as a spectogram
X = librosa.stft(x)
Xdb = librosa.amplitude_to_db(abs(X))
plt.figure(figsize=(14, 5))
librosa.display.specshow(Xdb, sr=sr, x_axis='time', y_axis='hz')
plt.title('Spectogram')
plt.colorbar()


# Visualizing Audio as Mel-Spectogram

file_location = audio_path
y, sr = librosa.load(file_location)
```

```
melSpec = librosa.feature.melspectrogram(y=y, sr=sr, n_mels=128)
melSpec_dB = librosa.power_to_db(melSpec, ref=np.max)
plt.figure(figsize=(10, 5))
librosa.display.specshow(melSpec_dB, x_axis='time', y_axis='mel', sr=sr, fmax=8000)
plt.colorbar(format='%+1.0f dB')
plt.title("MelSpectrogram")
plt.tight_layout()
plt.show()


import math
import json
import os
import librosa

# Set your constants
# MUSIC = '/content/drive/MyDrive/Data/genres_original'
DATASET_PATH = '/content/drive/MyDrive/Data/genres_original'
JSON_PATH = "data_10.json"
SAMPLE_RATE = 22050
TRACK_DURATION = 30  # measured in seconds
SAMPLES_PER_TRACK = SAMPLE_RATE * TRACK_DURATION

# Define the function to save MFCCs
def save_mfcc(dataset_path, json_path, num_mfcc=13, n_fft=2048, hop_length=512, num_segments=5):
    data = {
        "mapping": [],
        "labels": [],
        "mfcc": []
    }

    samples_per_segment = int(SAMPLES_PER_TRACK / num_segments)
    num_mfcc_vectors_per_segment = math.ceil(samples_per_segment / hop_length)

    # Loop through all genre sub-folders
    for i, (dirpath, dirnames, filenames) in enumerate(os.walk(dataset_path)):
        # Ensure we're processing a genre sub-folder level
        if '.ipynb_checkpoints' in dirpath:
            continue
        if dirpath is not dataset_path:
            # Save genre label (i.e., sub-folder name) in the mapping
```

```
            semantic_label = dirpath.split("/")[-1]
            data["mapping"].append(semantic_label)
            print("\nProcessing: {}".format(semantic_label))
            # Process all audio files in genre sub-dir
            for f in filenames:
                file_path = os.path.join(dirpath, f)
                # Load audio file
                signal, sample_rate = librosa.load(file_path, sr=SAMPLE_RATE)
                # Process all segments of audio file
                for d in range(num_segments):
                    # Calculate start and finish sample for current segment
                    start = samples_per_segment * d
                    finish = start + samples_per_segment
                    # Extract MFCC
                    mfcc = librosa.feature.mfcc(y=signal[start:finish], sr=sample_rate, n_mfcc=num_mfcc,
n_fft=n_fft, hop_length=hop_length)
                    mfcc = mfcc.T
                    # Store only MFCC feature with expected number of vectors
                    if len(mfcc) == num_mfcc_vectors_per_segment:
                        data["mfcc"].append(mfcc.tolist())
                        data["labels"].append(i-1)

                        print("{}, segment:{}".format(file_path, d+1))
        # print(i)

    # Save MFCCs to JSON file
    with open(json_path, "w") as fp:
        json.dump(data, fp, indent=4)


rm -rf find -type d -name .ipynb_checkpoints


# Run the function to save MFCCs
save_mfcc(DATASET_PATH, JSON_PATH, num_segments=6)


DATA_PATH = "./data_10.json"


def load_data(data_path):
```

```
    """Loads training dataset from json file.
        :param data_path (str): Path to json file containing data
        :return X (ndarray): Inputs
        :return y (ndarray): Targets
    """

    with open(data_path, "r") as fp:
        data = json.load(fp)

    X = np.array(data["mfcc"])
    y = np.array(data["labels"])
    z = np.array(data['mapping'])
    return X, y, z



    # print(X.shape)
    # print(y.shape)
    # print(z.shape)

def plot_history(history):
    """Plots accuracy/loss for training/validation set as a function of the epochs
        :param history: Training history of model
        :return:
    """

    fig, axs = plt.subplots(2)

    # create accuracy sublpot
    axs[0].plot(history.history["accuracy"], label="train accuracy")
    axs[0].plot(history.history["val_accuracy"], label="test accuracy")
    axs[0].set_ylabel("Accuracy")
    axs[0].legend(loc="lower right")
    axs[0].set_title("Accuracy eval")

    # create error sublpot
    axs[1].plot(history.history["loss"], label="train error")
    axs[1].plot(history.history["val_loss"], label="test error")
    axs[1].set_ylabel("Error")
    axs[1].set_xlabel("Epoch")
    axs[1].legend(loc="upper right")
    axs[1].set_title("Error eval")
```

```
    plt.show()


def prepare_datasets(test_size, validation_size):
    """Loads data and splits it into train, validation and test sets.
    :param test_size (float): Value in [0, 1] indicating percentage of data set to allocate to test split
    :param validation_size (float): Value in [0, 1] indicating percentage of train set to allocate to validation split
    :return X_train (ndarray): Input training set
    :return X_validation (ndarray): Input validation set
    :return X_test (ndarray): Input test set
    :return y_train (ndarray): Target training set
    :return y_validation (ndarray): Target validation set
    :return y_test (ndarray): Target test set
    :return z : Mappings for data
    """

    # load data
    X, y, z = load_data(DATA_PATH)

    # create train, validation and test split
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=test_size)
    X_train, X_validation, y_train, y_validation = train_test_split(X_train, y_train, test_size=validation_size)

    # add an axis to input sets
    X_train = X_train[..., np.newaxis]
    X_validation = X_validation[..., np.newaxis]
    X_test = X_test[..., np.newaxis]




    return X_train, X_validation, X_test, y_train, y_validation, y_test, z

  # # Get train, validation, test splits
  #   X_train, X_validation, X_test, y_train, y_validation, y_test, z = prepare_datasets(0.25, 0.2)

  #   # Check shapes
  #   print("X_train shape:", X_train.shape)
  #   print("X_validation shape:", X_validation.shape)
  #   print("X_test shape:", X_test.shape)
```

```
def build_model(input_shape):
    """Generates CNN model
    :param input_shape (tuple): Shape of input set
    :return model: CNN model
    """

    # build network topology
    model = keras.Sequential()

    # 1st conv layer
    model.add(keras.layers.Conv2D(32, (3, 3), activation='relu', input_shape=input_shape))
    model.add(keras.layers.MaxPooling2D((3, 3), strides=(2, 2), padding='same'))
    model.add(keras.layers.BatchNormalization())

    # 2nd conv layer
    model.add(keras.layers.Conv2D(32, (3, 3), activation='relu'))
    model.add(keras.layers.MaxPooling2D((3, 3), strides=(2, 2), padding='same'))
    model.add(keras.layers.BatchNormalization())

     # 3rd conv layer
    model.add(keras.layers.Conv2D(32, (2, 2), activation='relu'))
    model.add(keras.layers.MaxPooling2D((2, 2), strides=(2, 2), padding='same'))
    model.add(keras.layers.BatchNormalization())

    # flatten output and feed it into dense layer
    model.add(keras.layers.Flatten())
    model.add(keras.layers.Dense(64, activation='relu'))
    model.add(keras.layers.Dropout(0.3))

    # output layer
    model.add(keras.layers.Dense(10, activation='softmax'))

    return model



def predict(model, X, y):
    """Predict a single sample using the trained model
    :param model: Trained classifier
    :param X: Input data
```

```
    :param y (int): Target
    """

    # add a dimension to input data for sample - model.predict() expects a 4d array in this case
    X = X[np.newaxis, ...] # array shape (1, 130, 13, 1)

    # perform prediction
    prediction = model.predict(X)

    # get index with max value
    predicted_index = np.argmax(prediction, axis=1)

    # get mappings for target and predicted label
    target = z[y]
    predicted = z[predicted_index]

    print("Target: {}, Predicted label: {}".format(target, predicted))


# get train, validation, test splits
X_train, X_validation, X_test, y_train, y_validation, y_test, z = prepare_datasets(0.25, 0.2)

# create network
input_shape = (X_train.shape[1], X_train.shape[2], 1)
model = build_model(input_shape)

# compile model
optimiser = keras.optimizers.Adam(learning_rate=0.0001)
model.compile(optimizer=optimiser,
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

# train model
history = model.fit(X_train, y_train, validation_data=(X_validation, y_validation), batch_size=32,
epochs=30)

# plot accuracy/error for training and validation
plot_history(history)
# evaluate model on test set
```

```
test_loss, test_acc = model.evaluate(X_test, y_test, verbose=2)
print('\nTest accuracy:', test_acc)
```

```
from sklearn.metrics import accuracy_score, precision_recall_fscore_support

# Get overall accuracy
overall_accuracy = accuracy_score(y_test, y_pred)

# Get overall precision, recall, f1-score, and support
overall_precision, overall_recall, overall_fscore, overall_support = precision_recall_fscore_support(y_test, y_pred, average='weighted')

# Print overall metrics
print("Overall Metrics:")
print(f"Overall Accuracy: {overall_accuracy}")
print(f"Overall Precision: {overall_precision}")
print(f"Overall Recall: {overall_recall}")
print(f"Overall F1-Score: {overall_fscore}")
```

```
# pick a sample to predict from the test set
X_to_predict = X_test[100]
y_to_predict = y_test[100]

# predict sample
predict(model, X_to_predict, y_to_predict)
```

```
import h5py

# Function to extract and save weights
def save_weights(model, file_path):
    """Extracts and saves weights of the model to a file.
    :param model: Trained model
    :param file_path: File path to save the weights
```

```
    """
    # Extract weights
    weights = model.get_weights()

    # Save weights to file
    with h5py.File(file_path, 'w') as f:
        for i, weight in enumerate(weights):
            f.create_dataset('weight_' + str(i), data=weight)

# Assuming you have already trained the model, let's say 'model' is your trained model instance
# Define the file path where you want to save the weights
weights_file_path = "/content/drive/MyDrive/saved_models/model_weights.h5"

# Call the function to save weights
save_weights(model, weights_file_path)

print("Weights saved successfully to:", weights_file_path)
```